

GOOD MORNING!

早上好!

안녕하세요!

DAY 2



YESTERDAY BEFORE GOING HOME YOU DID ONE OF THE FOLLOWING:

- Dock your robot
- Perform **soft** shutdown by
 - PC Terminal
\$ ssh ubuntu@<AMR IP>
 - AMR Terminal
\$ sudo shutdown now

- TB4 battery continues to charge

Or

- Perform **soft** shutdown by
 - PC Terminal
\$ ssh ubuntu@<AMR IP>
 - AMR Terminal
\$ sudo shutdown now
- Remove TB4 from the Docking Station
- Power off Create 3

- TB4 battery is **NOT** charging



TO POWER ON:

TB4 IS CHARGING AND CONNECTED TO THE DOCKING STATION

- Remove TB4 from Docking Station
- Power off the Create 3
- Dock the TB4 to the Docking Station

TB4 IS NOT CHARGING AND DISCONNECTED FROM THE DOCKING STATION

- Dock the TB4 to the Docking Station



DAY 1 RECAP



2 PROJECTS

- Mini Project (Individual Team)
 - For learning techniques

차시	구분	세부사항
1	프로젝트 계획 및 환경 구축	시스템 개발 프로세스의 이해, 개발 환경 구축
2	기술 탐색 및 검증	AI VISION 기술 탐색 및 검증
3	기술 탐색 및 검증	AMR 제어 기술 탐색 및 검증
4	기술 탐색 및 검증	Mini project 완성 및 발표

2 PROJECTS

- Final Project (2 Teams in One)

차시	구분	세부사항
5	프로젝트 설계	외부 시스템 모니터 기술 탐색 및 검증 파이널 프로젝트 시스템 요구사항 설계 및 프로세스 정립
6	개발	기능 구현 및 Unit Test
7	개발	기능 구현 및 Unit Test
8	개발	통합 시스템 구축 및 테스트
9	개발	통합 시스템 구축 및 테스트
10	최종 프레젠테이션 및 시연	프로젝트 발표 및 시연, 산출물 정리, 기술 컨퍼런스

평가 항목 기준

② 평가 항목 기준

평가 항목	평가 내용	평가 방법	
행동 역량	교육과정 운영기간 동안 성실도/적극성 등 행동적인 요소 중심 평가	· 출결 현황 평가 · 과제 수행 현황 평가** · 수강 태도	정량 평가
개인 역량	교육 이수 결과로 지식, 기술 등을 얼마나 향상시키고 변화시켰는지 평가	· 교과목 정기 평가	
조직 역량	팀별 실습(훈련) 및 실무 프로젝트를 진행하는 동안 조직간 의사소통/협업 역량 등 평가	· 협업 능력 평가	
기술 역량	실무 프로젝트 및 자율 과제 프로젝트 결과물 평가	· 프로젝트 산출물 평가	

** 연습문항/과제문항은 제출 결과를 기반으로 P/F 평가만 이루어집니다.

교육생 조직 역량 평가

4. [ROKEY-7기] 교육생 조직 역량 평가

1. 팀 운영 및 의사결정(5점)

- 역할 분담과 일정 관리가 체계적으로 이루어졌는가

2. 의사소통 및 협업 태도(5점)

- 팀 내 소통이 원활하고 협업 분위기가 잘 형성되었는가
- 적극적으로 참여하며 맡은 역할을 책임감 있게 수행했는가

3. 문제 해결 및 개선 노력(5점)

- 문제 상황에 적절히 대응하고 해결했는가
- 피드백을 반영하여 지속적으로 개선하려는 노력이 있었는가

5점: 매우 우수

- 체계적이고 일관되게 잘 수행했고 모범 사례로 볼 수 있음

4점: 우수

- 전반적으로 잘 이루어졌고 개선 여지

3점: 보통

- 기본 기대 수준으로 큰 문제 없이 무난하게 수행됨

2점: 미흡

- 일부 부족한 부분이 있으며 개선이 필요함

1점: 매우 미흡

- 전반적으로 부족하며 기준에 크게 미달함

기술 평가

▶ 평가 항목

평가 항목	평가기준
기능 구현 완전성	* 교육에서 요구하는 기능이 모두 구현되어 있는지 여부를 평가한다.
기능 구현 정확성	* 구현된 모든 기능들이 정상적으로 동작하는지 여부를 평가한다.
동작 및 운용 안정성	* 산출물 운용시 안정적으로 동작하는지 여부를 평가한다.
입출력 데이터 이해도	* 데이터 입출력 방법 및 절차가 편리하고 기능 요구 내용에 적합한 지 여부를 평가한다.
기능 동작 지속성	* 장애/오류 발생 시에도 지속적인 동작/운영이 가능한지 여부를 평가한다.

PROJECT 평가

🔑 평가 지표

항목	하위 항목	점수
1. 비즈니스 요구 사항 작성		10
2. Detection Alert Module의 코딩 및 테스트 수행		10
3. AMR Controller Module의 코딩 및 테스트 수행		10
4. System Monitor Module 코딩 및 테스트 수행		10
5. Process Flow Diagram을 사용하여 시스템 설계를 생성		30
6. System Design Doc 완성도		10
7. 모든 모듈의 시스템 통합 및 테스트 수행		
	Map 상 장애물 회피, Dock Undock init 자동화 goal arrival	10
	Detection: Object detection, depth 변환, 이동	10
	Realtime 장애/오류 Handling Function	10
	Quality of System Monitor Completed	10
	Level of System Integrated	20
8. 최종 프로젝트 발표		10
TOTAL		150

DAY I

- Welcome
- Project Introduction
- Introduction to Project Development Process
- Business Requirement Development
- System Requirement Development
- System and Development environment Setup

DAY 2 (MINI PROJECT)

- Yolo객체 인식 모델 활용과 성능 평가 방법 이해
 - Custom Dataset과 Fine Tuning으로 자체 객체 인식 모델 구현 및 평가
 - (Optional)경량화 모델 등 개별 요구사항에 적합한 모델 탐색 및 성능 검증

DAY 2 (MINI PROJECT)

WEB-CAM 기반 객체 인식

- YOLOv8 기반 데이터 수집/학습/deploy (Detection Alert)
 - 감시용 데이터 수집(rc_car, dummy, 등)
 - 감시용 데이터 라벨링
 - YOLOv8 기반 학습
 - YOLOv8 Object Detection

AMR-CAM 기반 객체 인식

- AMR(Autonomous Mobile Robot) Turtlebot4 개발 환경 구축
- 로봇 개발 환경에 완성 모델 서빙 및 테스트 / 로봇 H/W, 제반 환경의 한계점 도출
 - Tracking 데이터 수집((rc_car, dummy, 등)
 - Tracking 데이터 라벨링
 - YOLOv8 기반 학습
 - YOLOv8 Object **Tracking**

DAY 3 (MINI PROJECT)

- Auto. Driving 시스템 학습
 - Digital Mapping of environment
 - Operate AMR (Sim. & Real)
 - Tutorial 실행
 - Detection, Depth and AMR 주행
 - 로봇 개발 환경에 적용 및 테스트 / 로봇 H/W, 제반 환경의 한계점 도출

TURTLEBOT4 시뮬레이션 DEMO

- SLAM과 AutoSLAM으로 맵 생성
- Sim.Tutorial 실행
- Detection, Depth and AMR 주행 example

DAY 3 (MINI PROJECT)

REAL ROBOT

- Manually operating the AMR (Teleops)
- autonomous driving 시스템 with obstacle avoidance
 - Digital Mapping of environment
 - Launching Localization, Nav2, and using Rviz to operate a robot
 - Goal Setting and Obstacle Avoidance using Navigation

TUTORIAL

- Turtlebot4 API를 활용한 Initial Pose Navigate_to Pose 구현
- Turtlebot4 API를 활용한 Navigate_Through_pose, Follow Waypoints 구현

HOW TO WORK TOGETHER

- Participate, Participate, Participate!!!
- No long emails or Kakaotalk, prefer face to face
- Be open to suggestions and idea
- Be proactive (적극적), take initiative (주도적)
- HOW is as important as WHAT
- Ask the right questions? (to **YOU, team** and me)
- Investigate/Research/Analyze

프로젝트 RULE

80/20 → 20/80

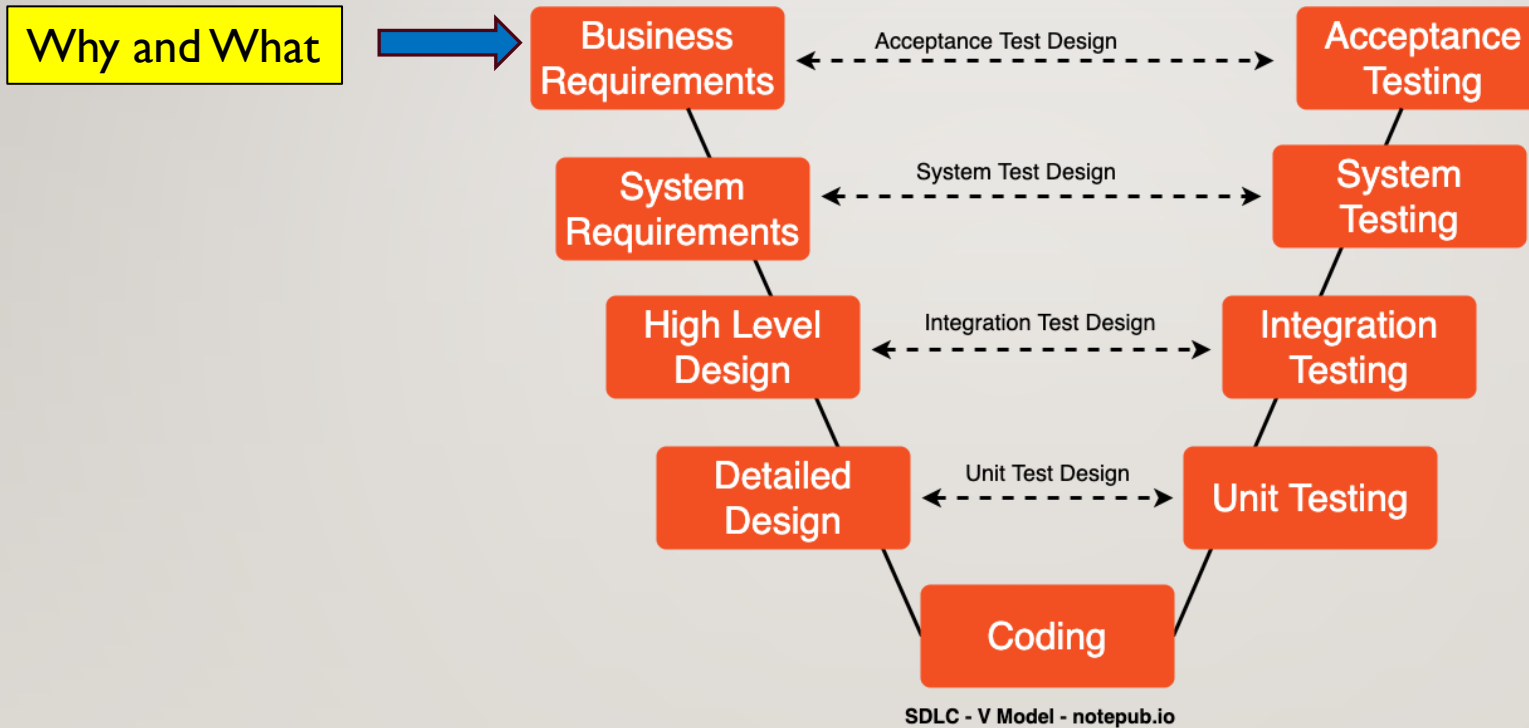
TEAMWORK AND PROJECT MANAGEMENT



PROJECT DEVELOPMENT IS A PROCESS

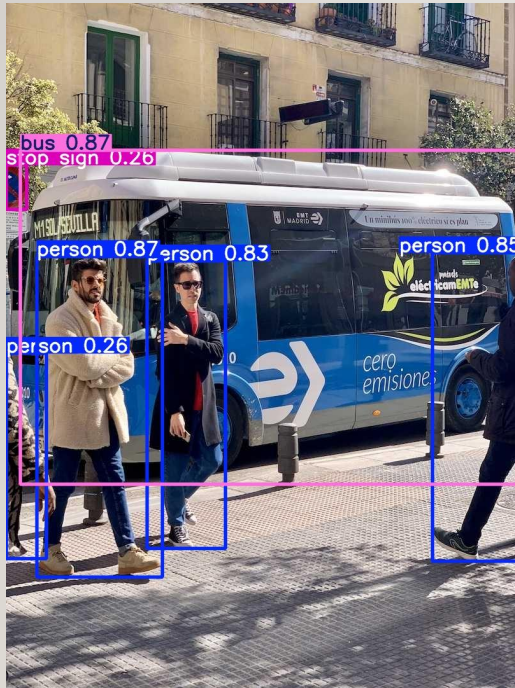


SW DEVELOPMENT PROCESS



ADVANCED TECHNIQUES THAT WE HAVE

- AI Object Detection

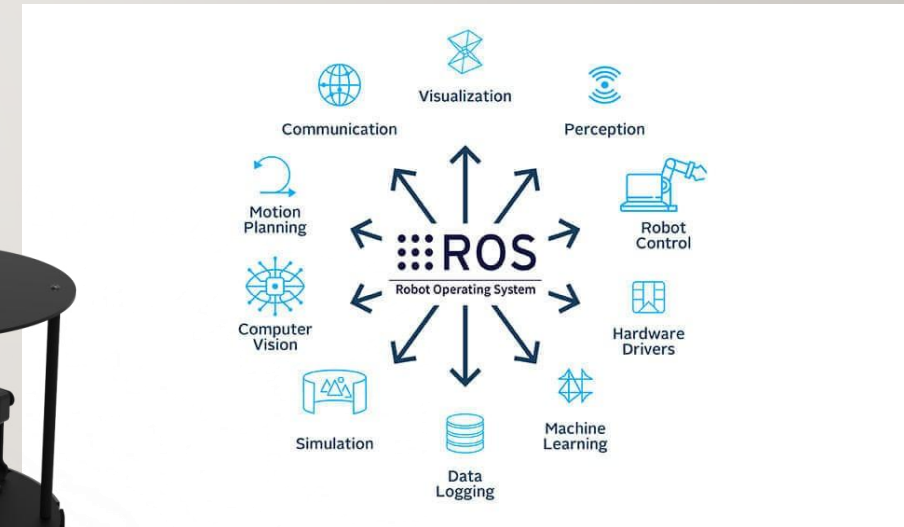


- AMR

- Navigation with obstruction avoidance
- Sensors



- ROS2



BRAINSTORM A SITUATION THAT WILL BENEFIT FROM **YOUR** SOLUTION

Must have measurable benefits. Search for them online



BRAINSTORMING RULES

- Every input is good input
- Do not critique inputs only seek to understand
- Organize inputs into logical groupings
- Sequence or show relationships as needed
- Use Posted Notes on Flip Chart



PRESENT THE SITUATIONS, SOLUTIONS AND MEASURABLE BENEFITS FROM THE BRAINSTORM

Identify the solution with highest measurable benefit

BUSINESS/SOLUTION REQUIREMENT STATEMENT (WHAT EXAMPLE)

- Commercial/Economical Requirements
 - ROI
 - cost reduction
 - Productivity
 - Revenue
 - Staffing
 - ...
- Technical/Functional Requirements
 - inspect aisles
 - detect defects
 - deliver objects
 - generate alerts
 - ...

BUSINESS/SOLUTION REQUIREMENT (WHAT EXAMPLE)

- **Business Requirements with Metrics:** The company aims to deploy a robotic system integrated with a deep learning model to automate quality inspection in manufacturing. The goal is to reduce human error by achieving 98% accuracy in defect detection and increase production efficiency by minimizing inspection time to under 2 seconds per item.
- 이 회사는 딥 러닝 모델과 통합된 로봇 시스템을 배포하여 제조 시 품질 검사를 자동화하는 것을 목표로 합니다. 목표는 결함 감지에서 **98%의 정확도**를 달성하여 인적 오류를 줄이고 검사 시간을 품목당 **2초 미만**으로 최소화하여 생산 효율성을 높이는 것입니다.

EXAMPLE OF POOR VS GOOD REQUIREMENTS

Poor Requirement

The robot should move fast.

The robot should detect people.

The robot should avoid obstacles.

The robot should have a good battery.

The robot should give alerts.

Better Requirement

The robot shall achieve a maximum speed of 0.5 m/s.

The robot shall detect standing adults within 5 m with at least 95% recall under normal indoor lighting.

The robot shall maintain a minimum clearance of 30 cm from obstacles while navigating.

The robot shall operate continuously for at least 8 hours between charges.

The robot shall issue visual and audible alerts within 1 second of detecting a critical fault.

CHARACTERISTICS OF HIGH-QUALITY REQUIREMENTS

- **Specific:** States exactly what is required.
- **Measurable:** Includes quantitative criteria where appropriate.
- **Achievable:** Technically feasible within project constraints.
- **Relevant:** Supports stakeholder or mission objectives.
- **Verifiable:** Can be validated through inspection, analysis, demonstration, or testing.
- **Traceable:** Has a unique identifier and can be linked to design elements, implementation, and test cases.

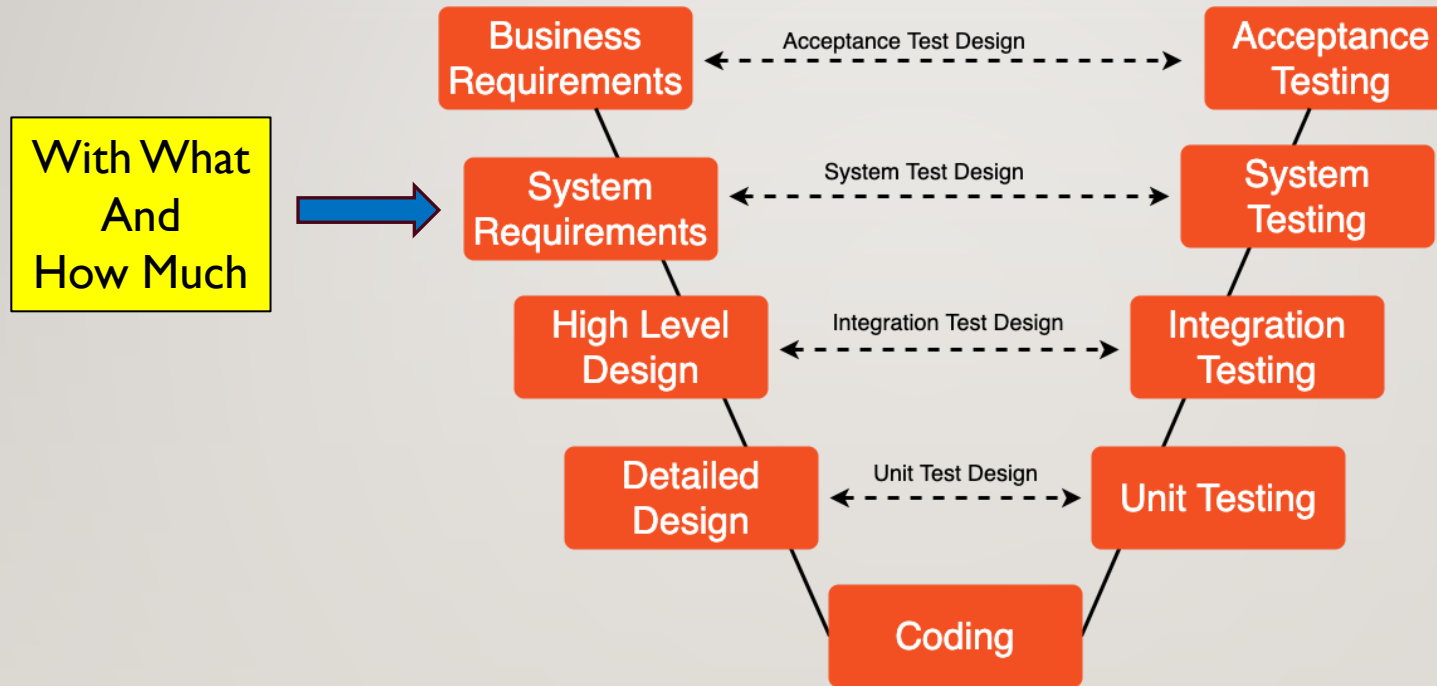
A. COMMERCIAL / ECONOMIC BUSINESS REQUIREMENTS

- BR-C01 — Labor Cost
 - The solution shall reduce warehouse inspection labor cost by at least 40% compared with the 2026 baseline.
- BR-C02 — Payback
 - The total project investment shall achieve a payback period of 24 months or less.
- BR-C03 — Productivity
 - The solution shall increase the number of completed inspections per employee-hour by at least 200%.
- BR-C04 — Operating Cost
 - Annual operating and maintenance cost shall not exceed ~~W~~\$50 million per warehouse.

B. TECHNICAL / OPERATIONAL BUSINESS REQUIREMENTS

- BR-T01 — Inspection Coverage
 - The solution shall inspect 100% of designated warehouse aisles at least once every four hours.
- BR-T02 — Defect Detection
 - The solution shall detect designated pallet defects with $\geq 95\%$ precision and $\geq 90\%$ recall under the defined warehouse operating conditions.
- BR-T03 — Alerting
 - The solution shall notify the responsible operator within 5 seconds after identifying a critical safety condition.
- BR-T04 — Mission Completion
 - The solution shall successfully complete $\geq 98\%$ of scheduled inspection missions without human intervention.
- BR-T05 — Availability
 - The inspection service shall be operational for $\geq 99.5\%$ of scheduled warehouse operating hours.

SW DEVELOPMENT PROCESS



SDLC - V Model - notepub.io

SYSTEM/TECHNICAL REQUIREMENT (EXAMPLE)

Technical Requirements and Metrics:

- 1. Deep Learning Models:** Train a CNN to achieve at least 98% accuracy on defect detection in validation datasets.

검증 데이터 세트에서 결함 감지에 대해 최소 98%의 정확도를 달성하도록 CNN을 훈련시킵니다.

- 2. Robotics Hardware:** Ensure the robot processes images and delivers results within 2 seconds per item, with 99.9% system uptime.

로봇이 99.9%의 시스템 가동 시간으로 항목당 2초 이내에 이미지를 처리하고 결과를 제공하도록 보장합니다.



SRD ANSWERS: WHAT DO WE NEED TO ACHIEVE THOSE NUMBERS?

- Suppose BR-T01 requires:
 - Inspect 100% of aisles every four hours.
- Engineering analysis might determine that this requires:

<u>SR ID</u>	<u>With What</u>	<u>How Much / How Many</u>
SR-01	TurtleBot4	10 robots
SR-02	Battery	≥8-hour runtime
SR-03	Navigation	Nav2, ≥0.5 m/s operational speed
SR-04	Network	Wi-Fi 6, ≤50 ms latency

HOMEWORK CHECK



SELECT **ONE** SOLUTION IDEA FROM THE BRAINSTORM AND UPDATE:

BUSINESS REQUIREMENT

- Research and Update
 - Situation Analysis with metrics
 - Pain Point/Need Analysis with metrics
- Define
 - Project/solution scope/out of scope
 - Functional Requirements with target/limits
 - Acceptance Criteria

SYSTEM REQUIREMENT

- Define
 - HW, OS, SW, and Networking requirements
 - Functional Requirements
 - Acceptance Criteria

REVIEW AMR (TURTLEBOT4) E-MANUAL

- [Features · User Manual](#)
- <https://turtlebot.github.io/turtlebot4-user-manual/overview/features.html>



PLEASE REVIEW YOUR WORK FROM EARLIER ONLINE CLASS

- Yolo obj. Det. Vs. Yolo Tracking
 - [Object Detection - Ultralytics YOLO Docs](#)
 - [Track - Ultralytics YOLO Docs](#)
 - [Model Training with Ultralytics YOLO - Ultralytics YOLO Docs](#)
- Yolo
 - Data Labelling (ex: Labellmg/roboflow)
 - Data pre-processing for YoloV8 Training
 - YoloV8 training to create .pt file
 - Using .pt file to predict/inference
- ROS
 - colcon build
 - Node, Topic, Service, Action, Interface, etc. coding

ROS EXERCISE I

Create a ROS2 Package with these publisher and subscribers

```
2_0_a_image_publisher.py
2_0_b_image_subscriber.py
2_0_c_data_publisher.py
2_0_d_data_subscriber.py
```

Try these CLI

```
$ ros2 run rqt_graph rqt_graph
$ ros2 node list
$ ros2 node info <node_name>
$ ros2 topic list
$ ros2 topic info <topic_name>
$ ros2 topic echo <topic_name>
$ ros2 interface list
$ ros2 interface show
  <package_name>/msg/<MessageName>
```

ROS EXERCISE 2

DAY 2 - AI VISION (YOLO)

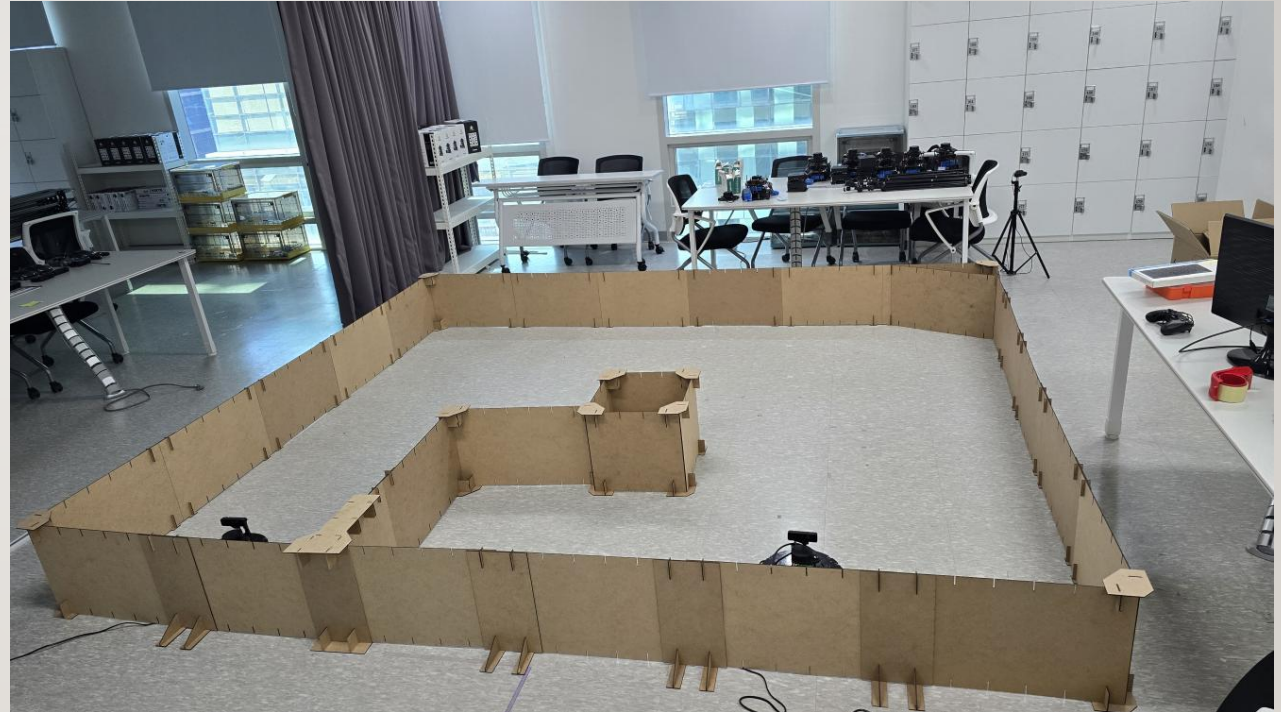
Aa 이름

⚙ status

● Homework_삐뽀삐뽀 소리내기 노드 만들기

● 완료

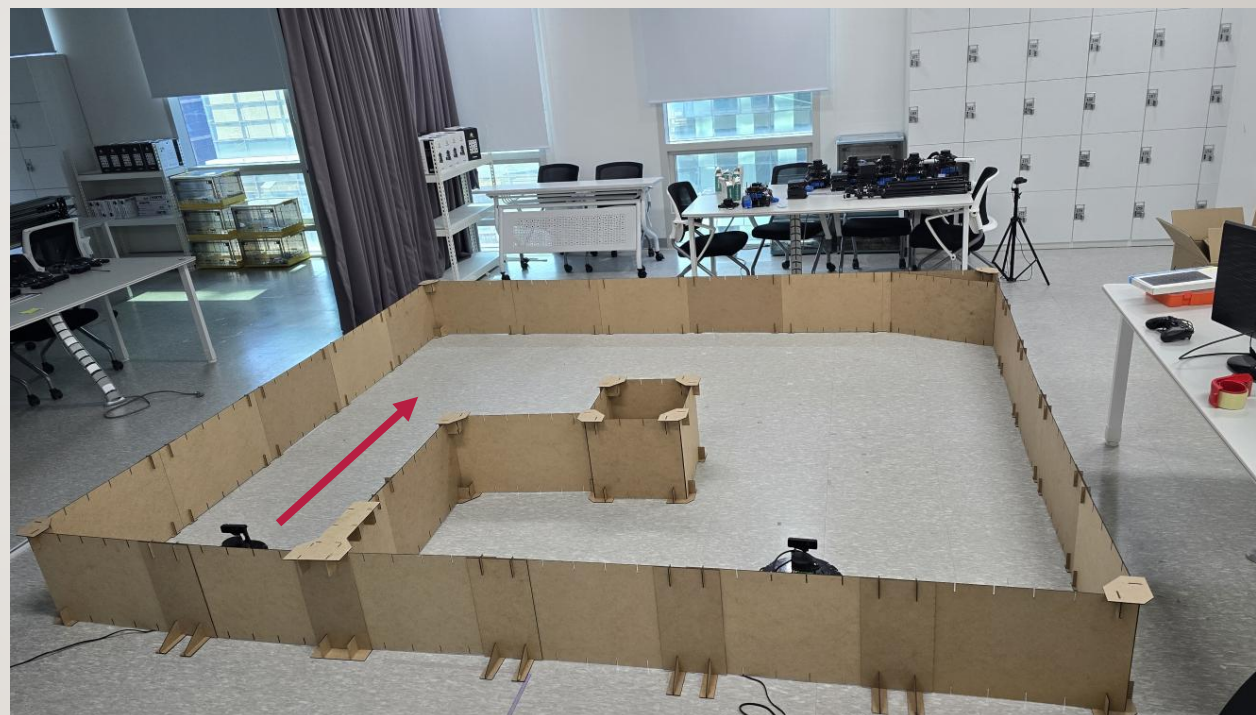
MINI PROJECT DESCRIPTION



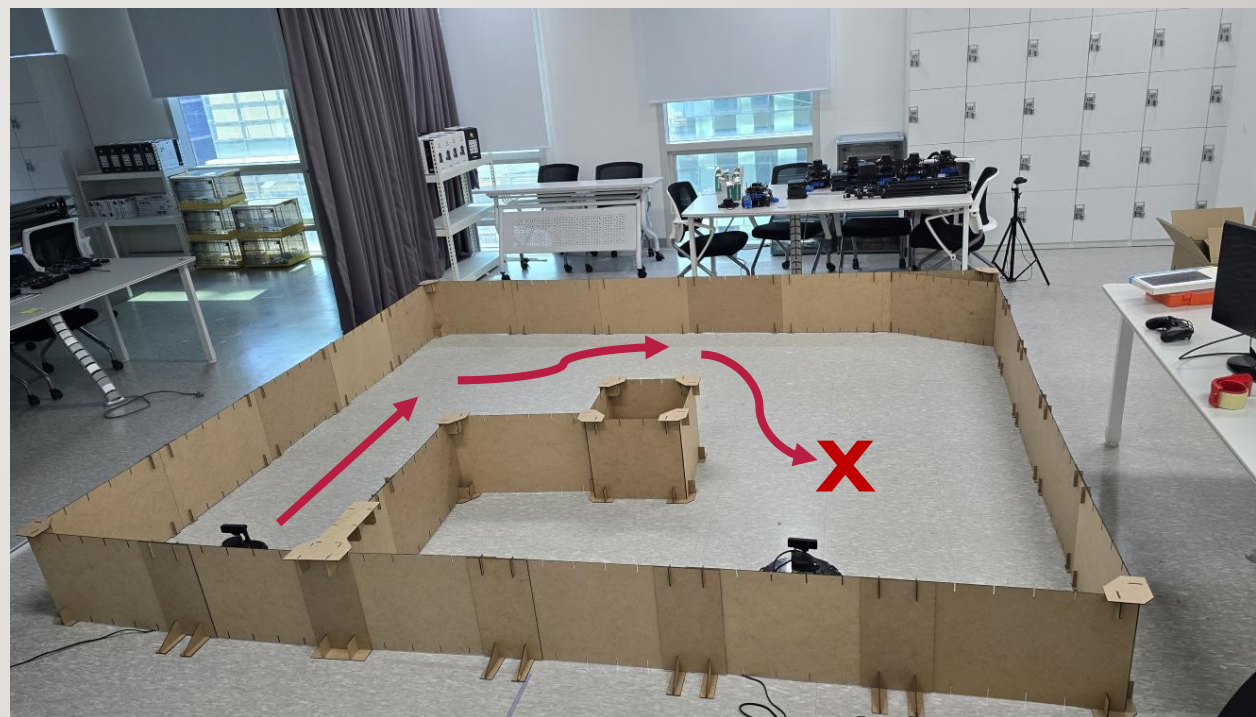
DETECTION ALERT



START



NAVIGATE TO A POSITION

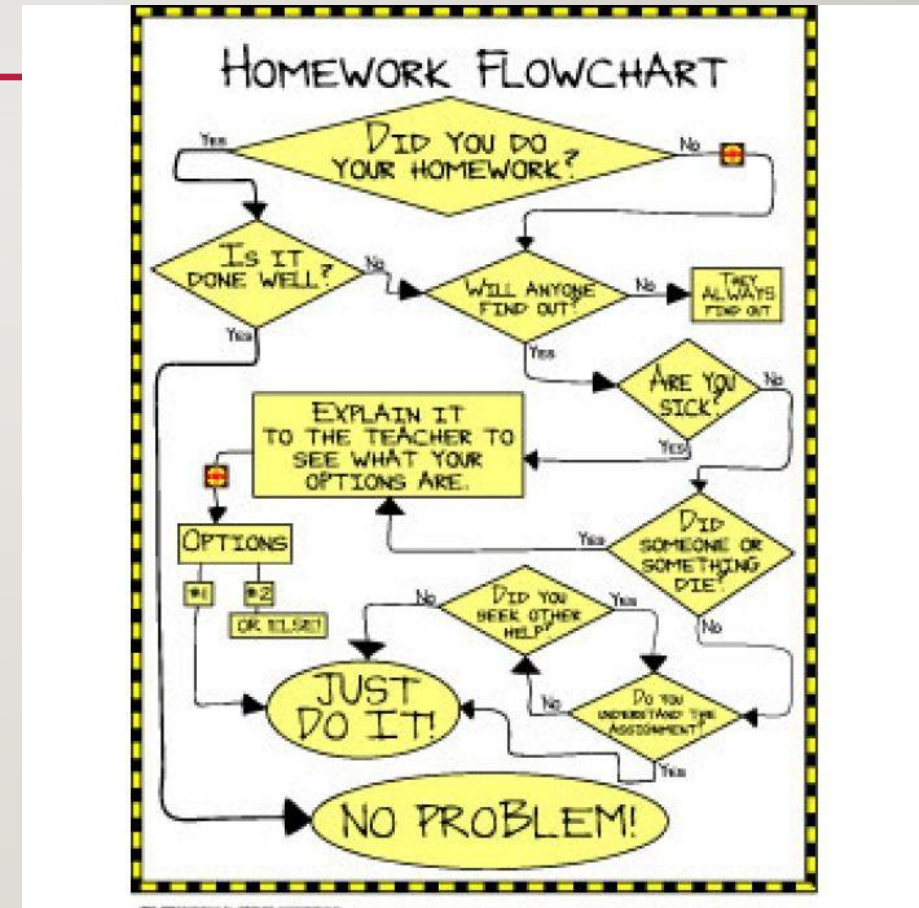


TRACK & APPROACH



VISUALIZATION – SCENARIO PROCESS DIAGRAMS

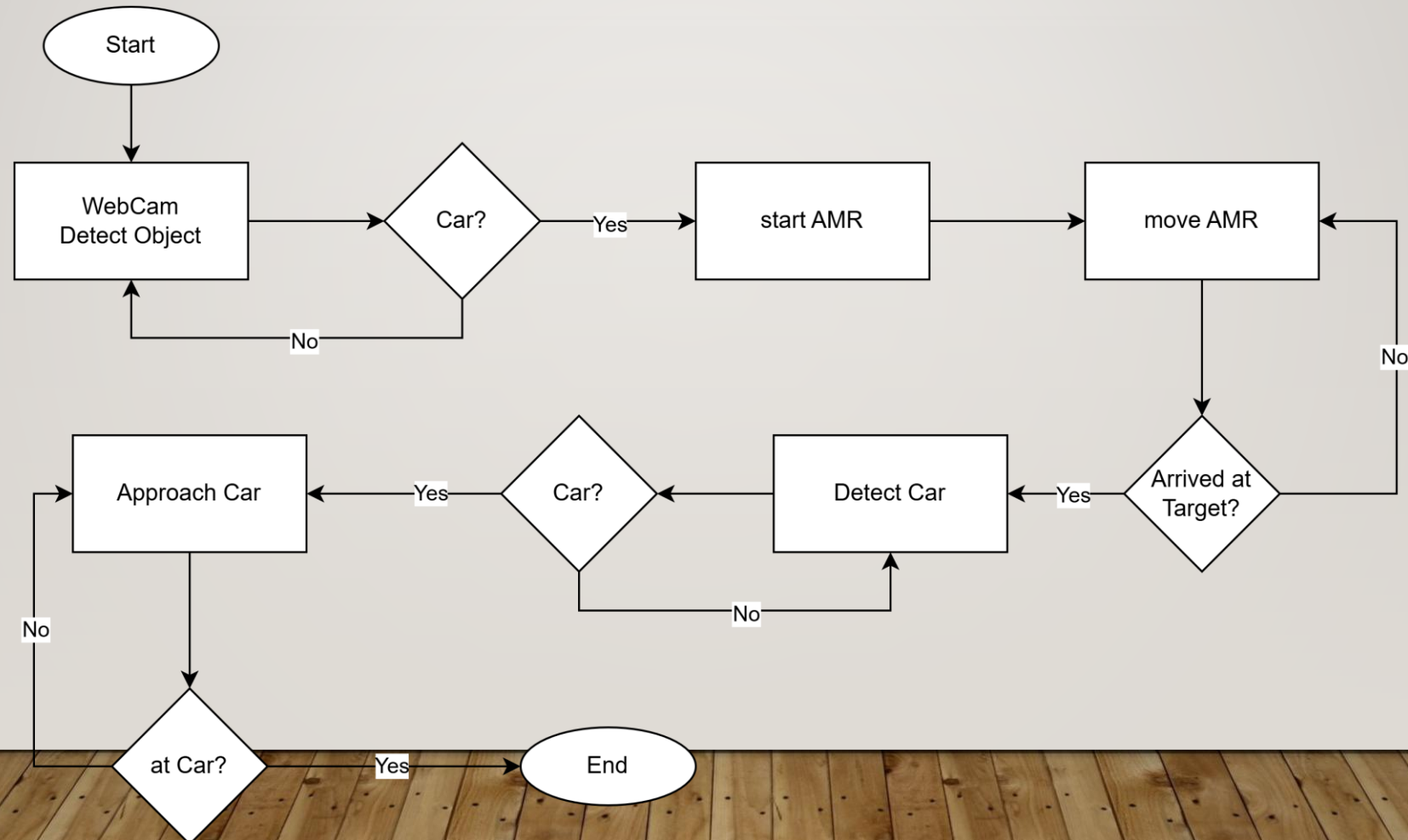
- As-Is Functional Process Diagram
 - Current states
- To-Be Functional Process Diagram
 - Future states
- [Untitled Diagram - draw.io](#)
- <https://app.diagrams.net/>



DEVELOP MINI PROJECT SCENARIO (USE-CASE) PROCESS DIAGRAM

Using the posted notes and flipchart as needed

MINI PROJECT – EXAMPLE SCENARIO DIAGRAM



MINI PROJECT: WITH WHAT



YOUR PROJECT ENVIRONMENT



BASE HW/OS

- PC

- Ubuntu 24.04
- ROS2 Jazzy
- USB Camera



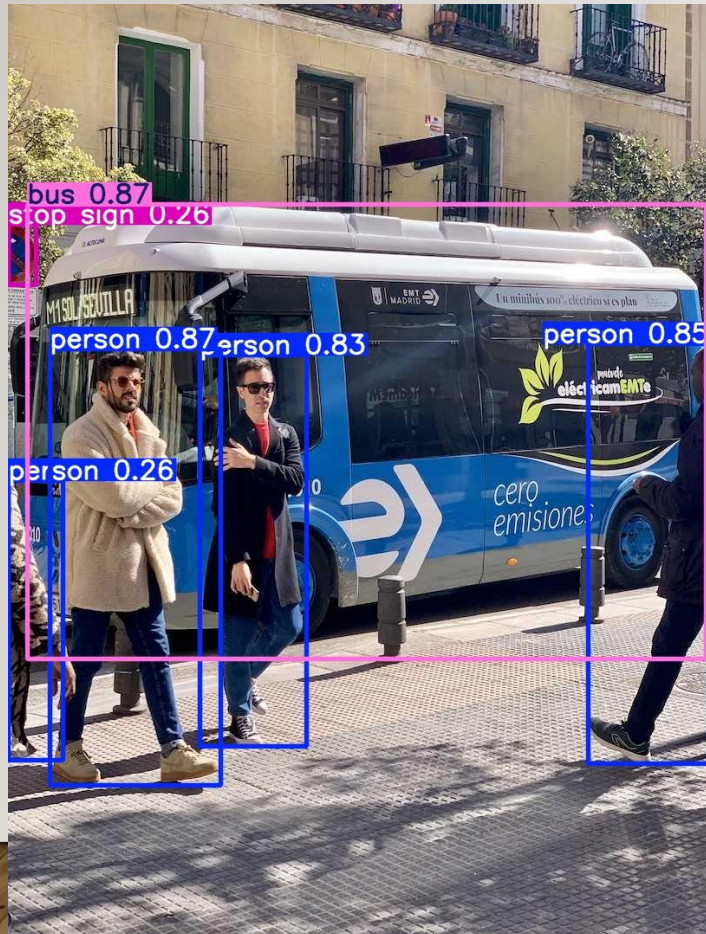
- Network
 - Wifi

- AMR

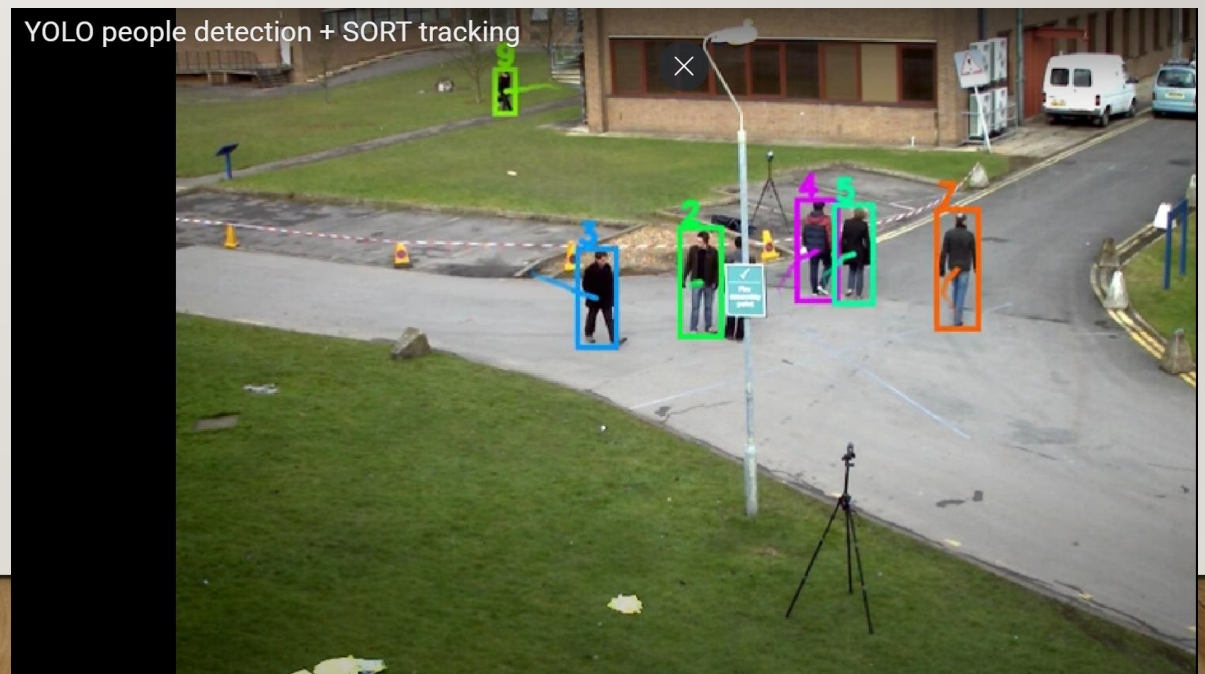
- TurtleBot4
- ROS2 Jazzy
- Ubuntu 24.04



YOLO OBJ. DET. VS. YOLO TRACKING



- [Track - Ultralytics YOLO Docs](#)
 - [\(469\) YOLO people detection + SORT tracking – YouTube](#)
 - [Bing Videos](#)



KEY SUBSYSTEM (MODULES) TO DEVELOP

- Detection Alert
 - Camera Capture
 - Object Detection
 - Send messages to other subsystems
- AMR Controller
 - Receive messages and act accordingly
 - Move using (SLAM) with Obstruction avoidance
 - Target Acquisition (Obj. Det.) and Tracking
 - Follow target using camera and motor control

SYSTEM AND DEVELOPMENT ENVIRONMENT SETUP



PROJECT SW SETUP ALREADY DONE

DAY 1 - Setup/Development ...

Aa 이름

☰ ⬆ ⚡ ✨ 🔍 ↗

● 평가 지표

☀ 상태

● PC 환경 구성

● 시작 전

● Turtlebot4 SW Setup

● 시작 전

● YOLO Setup

📄 열기

● 시작 전

● User PC Single Robot Network Setup

● 시작 전

● PC .bashrc 구성 Example

● 시작 전

프로젝트 RULE NUMBER ONE!!!

Are we having
Fun???



DAY 2



KEY SUBSYSTEM (MODULES) TO DEVELOP

- Detection Alert

- Camera Capture
- Object Detection
- Send messages to other subsystems

- AMR Controller

- Receive messages and act accordingly
- Move using (SLAM) with Obstruction avoidance
- Target Acquisition (Obj. Det.) and Tracking
- Approach target using camera and motor control

PERFORM DATA COLLECTION FOR DETECTION ALERT



AIM OF THE GOOD OBJ. DET. TRAINING SET

- Training images matches the real inference images as much as possible
 - Lighting
 - Dimensions
 - FOV
 - Backgrounds
 - Objects
 - ...
- For RGB and Depth and ...

COLLECTION IMAGES FROM WEBCAM

- Image Capture (WEBCAM)



```
2_1_a_capture_wc_image.py  
2_1_b_cont_capture_wc_image.py  
2_1_c_capture_wc_thread.py
```

- Resolution vs Dimension

COLLECTION IMAGES FROM AMR CAMERA



- Undock to see camera topics
- Are all camera topics available?
- Which image topic to use?
- Which dimensions and resolution?
- Are Depth and RGB pixel aligned?
- ...

NEED TO UNDOCK AMR TO ACTIVATE IMAGE TOPICS TO PUBLISH

UNDOCK

\$ ros2 topic list

Check the list

\$ ros2 action send_goal
/robot<n>/undock
irobot_create_msgs/action/Undock
“{”

\$ ros2 topic list

Check the list and compare

DOCK

\$ ros2 action send_goal /robot<n>/dock
irobot_create_msgs/action/Dock “{”

RGB CAMERA



DAY 2 - AI VISION (YOLO)

Aa 이름

⚙ status



Homework_삐뽀삐뽀 소리내기 노드 만들기



완료



RGB Camera



완료

DIMENSIONS AND RESOLUTION AND FPS

Supported `i_resolution` values (RGB):

Resolution Keyword	Width × Height	Notes
1080P	1920 × 1080	Default, high-res
720P	1280 × 720	Medium-res
800P	1280 × 800	Slightly taller
480P	640 × 480	✓ Ideal for alignment with stereo
400P	640 × 400	Wide, cropped top/bottom
320P	640 × 360	Lower-res
240P	320 × 240	Very low-res, fast

```
i_usb_speed: SUPER_PLUS
rgb:
  i_board_socket_id: 0
  i_fps: 30.0
  i_height: 720
  i_interleaved: false
  i_max_q_size: 10
  i_preview_size: 320
```



Use `rqt` to check and compare the different image topics

USING DEPTH

DAY 2 - AI VISION (YOLO)

Aa 이름

status

Homework_삐뽀삐뽀 소리내기 노드 만들기

완료

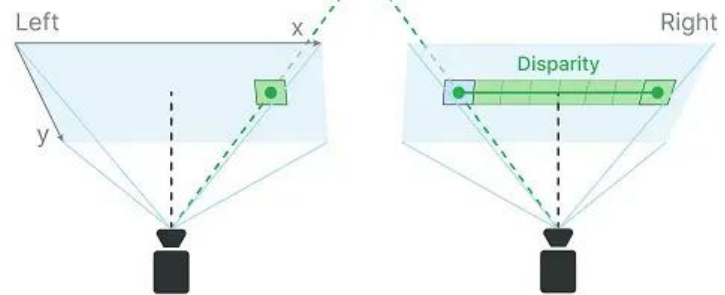
RGB Camera

완료

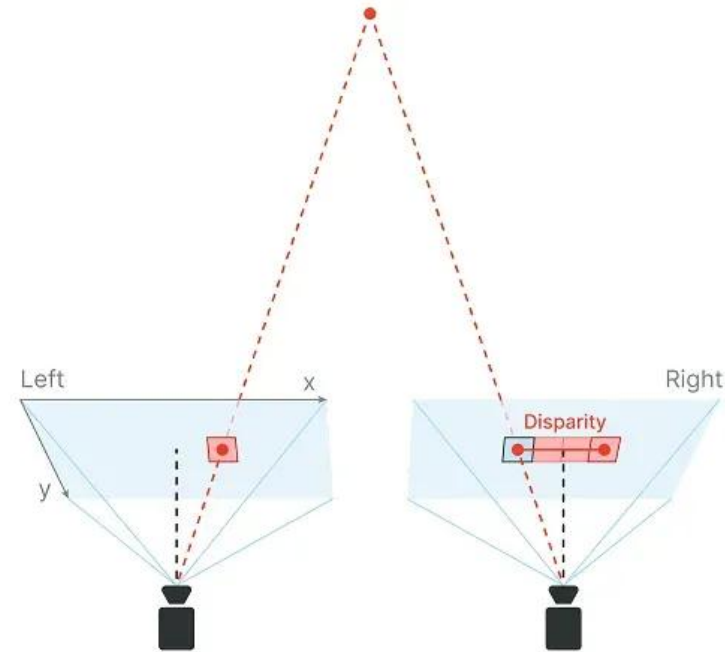
Depth Camera

완료

DEPTH INTRO



Stereo camera pair



Stereo camera pair

UPDATING THE OAKD CONFIG (ROBOT)

ON TURTLEBOT4:

```
$ cd  
  /opt/ros/jazzy/share/turtlebot4_bringup/config
```

```
$ sudo cp oakd_pro.yaml oakd_pro_orig.yaml
```

```
$ sudo cp oakd_pro_new.yaml oakd_pro.yaml
```

```
$ turtlebot4-service-restart
```

Or

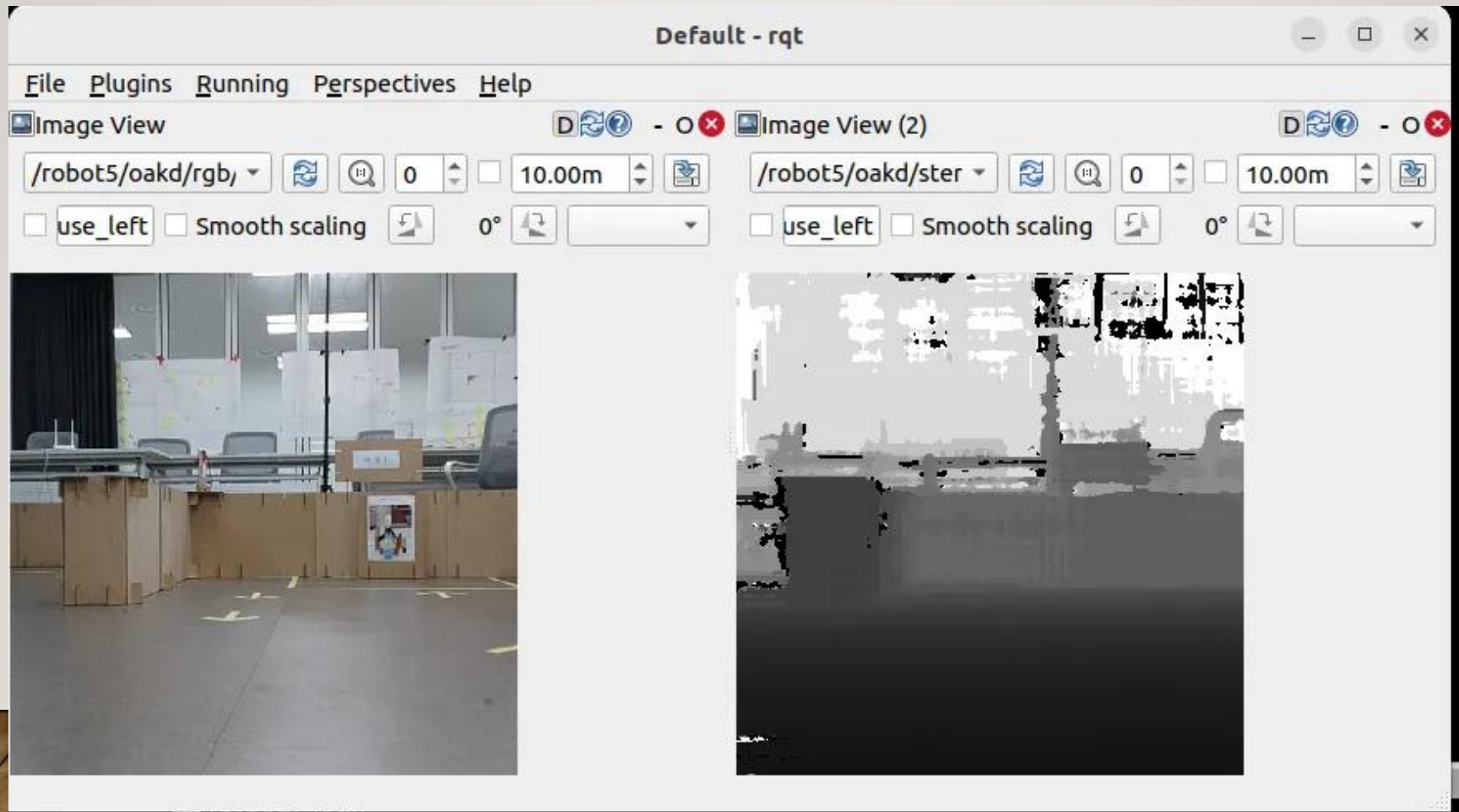
```
$ sudo reboot
```

```
1 /oakd:  
2   ros_parameters:  
3     use_sim_time: false  
4  
5   camera:  
6     i_enable_imu: false  
7     i_enable_ir: false  
8     i_floodlight_brightness: 0  
9     i_laser_dot_brightness: 100  
10    i_nn_type: none  
11    i_pipeline_type: RGBD # RGB + Depth  
12    i_usb_speed: SUPER_PLUS  
13  
14    rgb:  
15      i_board_socket_id: 0  
16      i_width: 640  
17      i_height: 480  
18      i_fps: 30.0  
19      i_enable_preview: true  
20      i_interleaved: false  
21      i_low_bandwidth: true  
22      i_publish_topic: true  
23      i_resolution: 480P # sets 640x480 internally  
24  
25  
26    stereo: # [x] Required to enable depth  
27      i_board_socket_id: 1  
28      i_fps: 30.0
```

WHICH IMAGE TOPIC TO USE?

- /oakd/rgb/preview/image_raw
 - /oakd/rgb/image_raw
 - /oakd/rgb/image_raw/compressed
 - /oakd/stereo/image_raw
 - ...
-
- ***** not all of the topics are visible, initially**
-
- EXERCISE
 - **Use rqt to view different image topics**
 - **Are all the topics viewable?**

RGB AND DEPTH PIXEL TO PIXEL ALIGNMENT – WHY NEED IT?



EXERCISE: ACHIEVE ALIGNED **FOV AND DIMENSION** FOR BOTH DEPTH AND RGB

Follow instruction on the notion to update the oakd config file.

Find config settings that will give same FOV for both Depth and RGB

Use rqt to view the topics



CODING HINTS

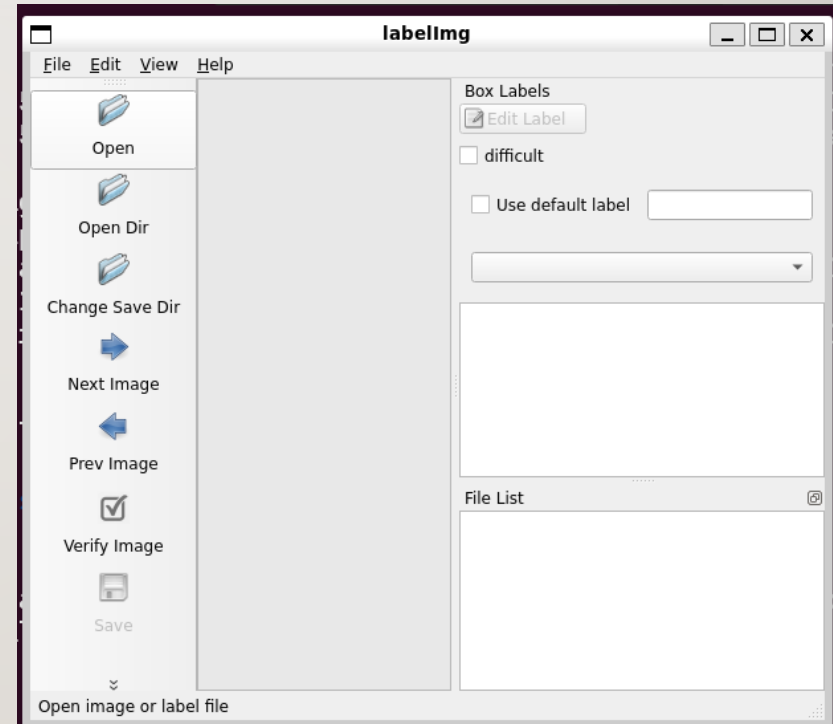
- Image Capture
- Image Capture (AMR)



```
2_1_a_capture_wc_image.py
2_1_b_cont_capture_wc_image.py
2_1_c_capture_wc_thread.py
2_1_d_capture_image.py
2_1_e_cont_capture_image.py
```

DATA LABELLING

- Image Capture
- Data Labelling
 - Goto the /labellmg/data/ directory
 - Rename the predefined_classes.txt



DATA LABELLING

- Data Labelling : Labellmg

라벨링 순서

1. 이미지파일 불러오기 (Open Dir)
2. 저장형식 변경 (PascalVOC, YOLO)
3. 이미지 선택
4. 바운딩 박스 그리기(create rectbox)
5. Class 지정
6. 저장경로 생성 및 변경(Change Save Dir)
7. 저장(Save)

단축키

Ctrl + u	Load all of the images from a directory
Ctrl + r	Change the default annotation target dir
Ctrl + s	Save
Ctrl + d	Copy the current label and rect box
Ctrl + Shift + d	Delete the current image
Space	Flag the current image as verified
w	Create a rect box
d	Next image
a	Previous image
del	Delete the selected rect box
Ctrl++	Zoom in
Ctrl--	Zoom out
↑→↓←	Keyboard arrows to move selected rect box

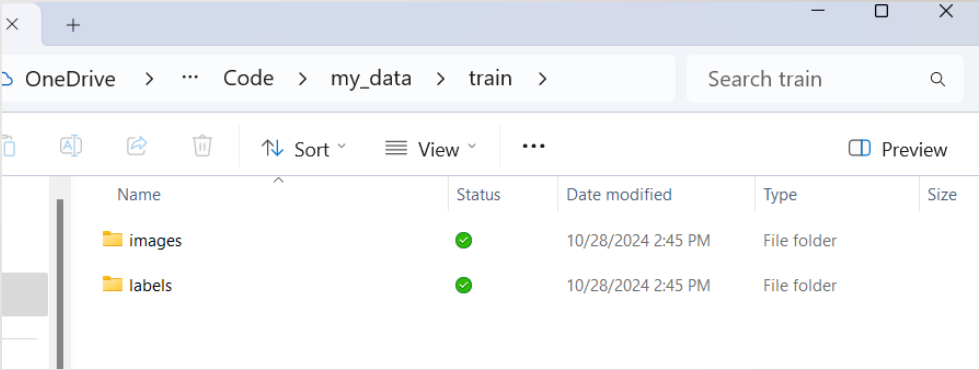
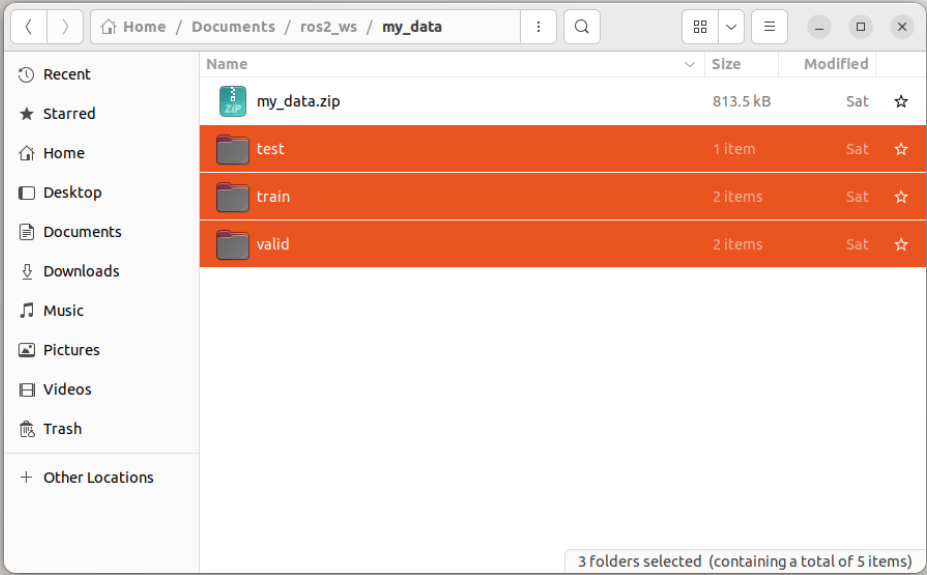
PRE-PROCESSING

- Image Capture
- Data Labelling
- Data Preprocessing



```
2_1_a_capture_wc_image.py
2_1_b_cont_capture_wc_image.py
2_1_c_capture_wc_thread.py
2_1_d_capture_image.py
2_1_e_cont_capture_image.py
2_3_a_create_data_dirs.py
2_3_b_move_image.py
2_3_c_move_labels.py
```

ZIP TRAIN DATA SET



PERFORM YOLO TRAINING & INFERENCE



CODING HINTS

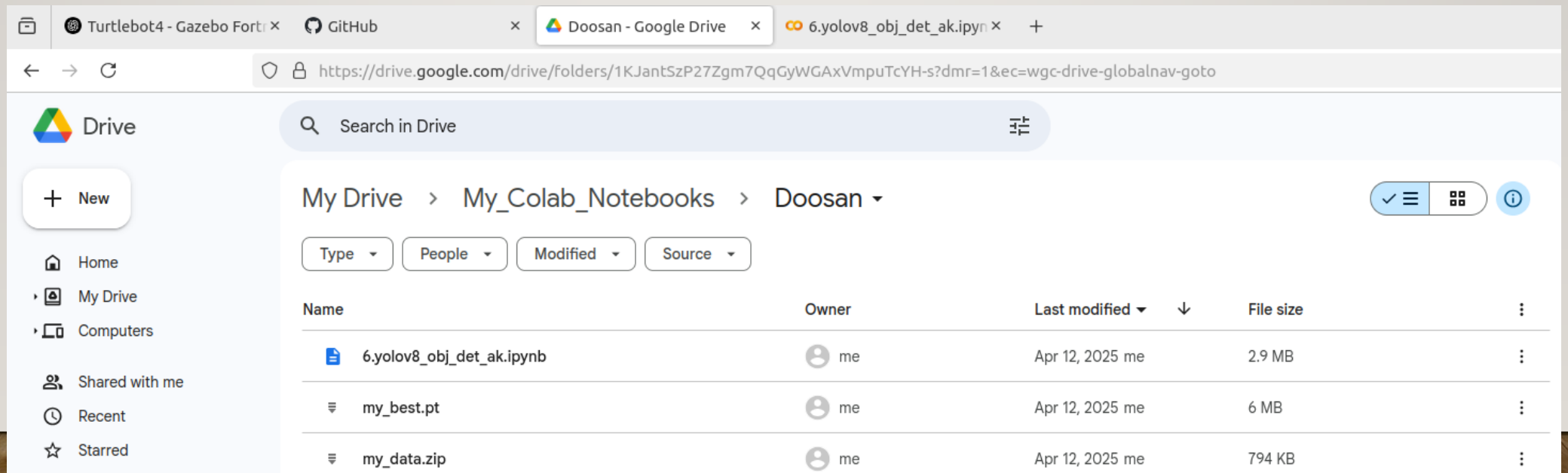
- Image Capture
- Data Labelling
- Preprocessing
- Yolo8 Object Det (Training)



```
2_3_a_create_data_dirs.py
2_3_b_move_image.py
2_3_c_move_labels.py
2_4_a_yolov8_obj_det_ak.ipynb
2_4_b_gpu_test.py
2_4_c_compare_yolo.py
```

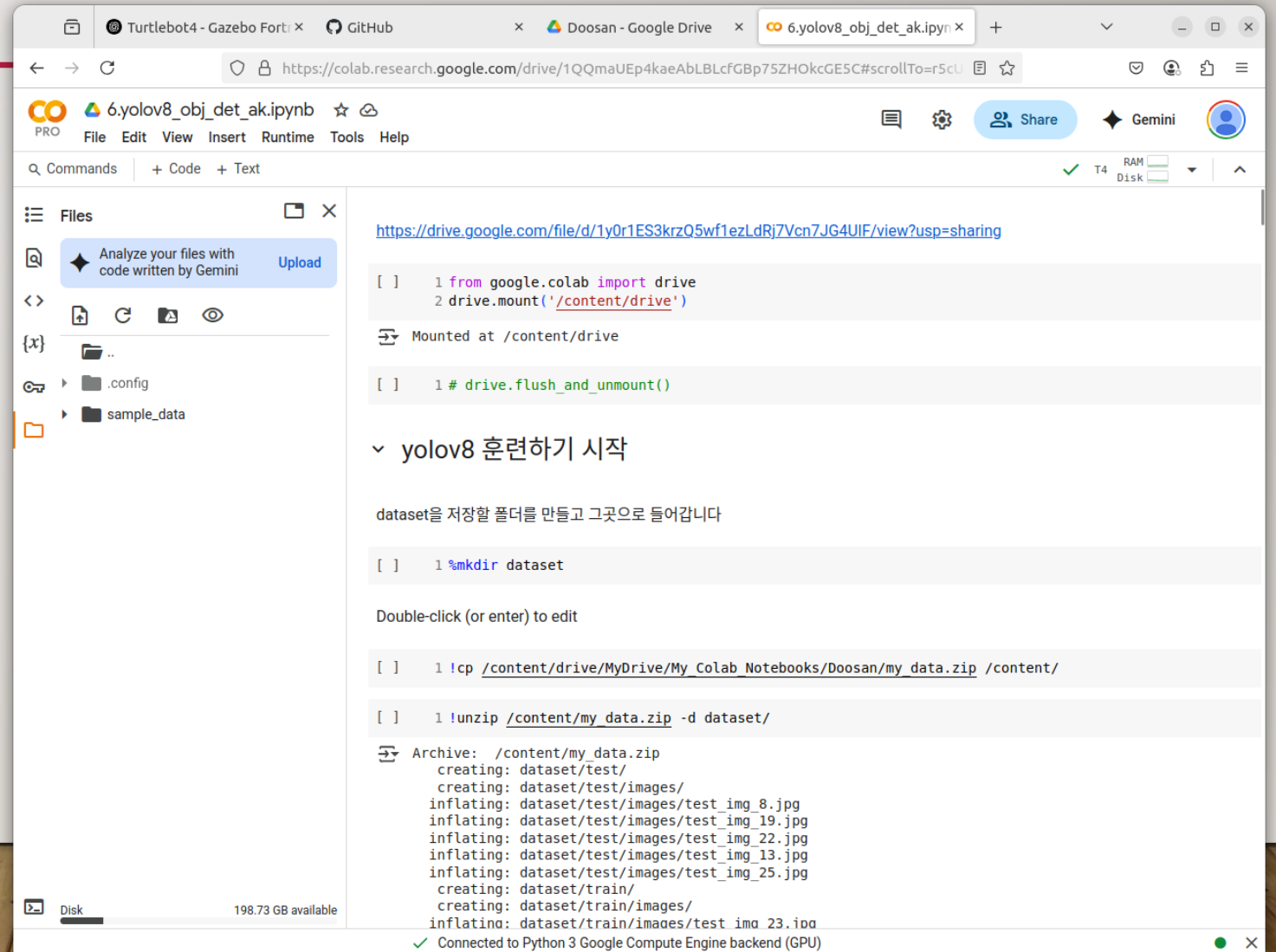
USING GOOGLE COLLAB.TO CREATE CUSTOM MODEL

- Move the files to google drive
 - my_data.zip
 - yolov8.obj.det.ak.ipynb



USING GOOGLE COLLAB.TO CREATE CUSTOM MODEL

- Move the training script to google collab. and execute line by line
- Model Training with Ultralytics YOLO - Ultralytics YOLO Docs



```
1 from google.colab import drive
2 drive.mount('/content/drive')

Mounted at /content/drive

1 # drive.flush_and_unmount()

yolov8 훈련하기 시작

dataset을 저장할 폴더를 만들고 그곳으로 들어갑니다

1 %mkdir dataset

Double-click (or enter) to edit

1 !cp /content/drive/MyDrive/My_Colab_Notebooks/Doosan/my_data.zip /content/

1 !unzip /content/my_data.zip -d dataset/

Archive: /content/my_data.zip
creating: dataset/test/
creating: dataset/test/images/
inflating: dataset/test/images/test_img_8.jpg
inflating: dataset/test/images/test_img_19.jpg
inflating: dataset/test/images/test_img_22.jpg
inflating: dataset/test/images/test_img_13.jpg
inflating: dataset/test/images/test_img_25.jpg
creating: dataset/train/
creating: dataset/train/images/
inflating: dataset/train/images/test_img_23.jpg

198.73 GB available

Connected to Python 3 Google Compute Engine backend (GPU)
```

REQUIRED RESEARCH

1. object detection이 아닌 segmentation, pose, obb 등을 활용할 수 있을까?
2. 왜 **XX** 모델을 선정하였습니까?
 - yolo 다른 버전과 비교 분석
 - Hyper Parameters?
 - Use Key Metrics to compare
 - Accuracy vs Precision
 - Precision & Recall
 - mAP
 - F1 Confidence
 - ...

CLASSIFICATION VS. DETECTION

CLASSIFICATION

- Every sample has one ground-truth label
- Every sample gets one prediction
- TN (true negatives) are well-defined
- Example:
 - 100 images → 100 predictions → clean denominator
- Accuracy

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

DETECTION

- Does not include background (TN)
- what the model predicted (TP + FP)
- what the model missed (FN)
- Precision/Recall

PRECISION AND RECALL

Definitions

$$Precision = \frac{TP}{TP+FP}$$

$$Recall = \frac{TP}{TP+FN}$$

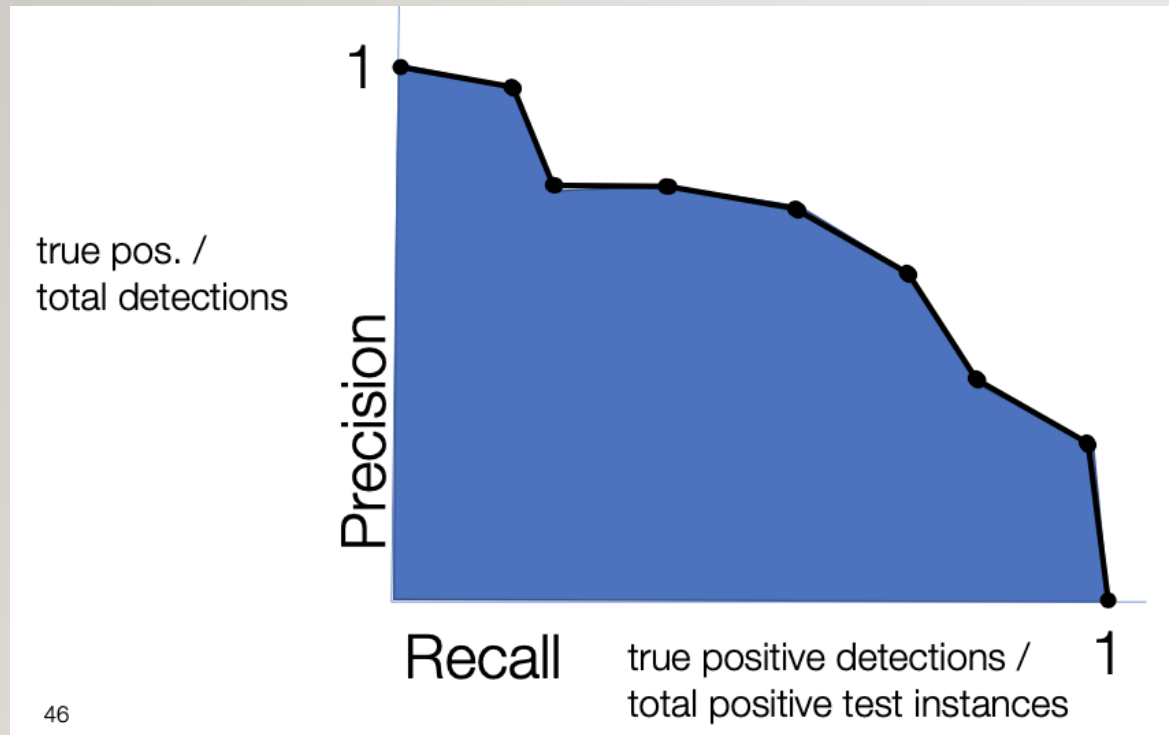
Plain meaning

- **Precision** → “When I detect something, am I right?”
- **Recall** → “Did I find all the real objects?”

PRECISION VS RECALL EXAMPLE

- Ground truth: 5 objects
- Model A (aggressive)
 - predicts 10 boxes
 - 5 correct, 5 wrong
 - precision = $5 / 10 = 0.50$
 - recall = $5 / 5 = 1.00$
 - Finds everything, but messy.
- Model B (conservative)
 - predicts 2 boxes
 - 2 correct
 - precision = $2 / 2 = 1.00$
 - recall = $2 / 5 = 0.40$
 - Very clean, but misses most objects.

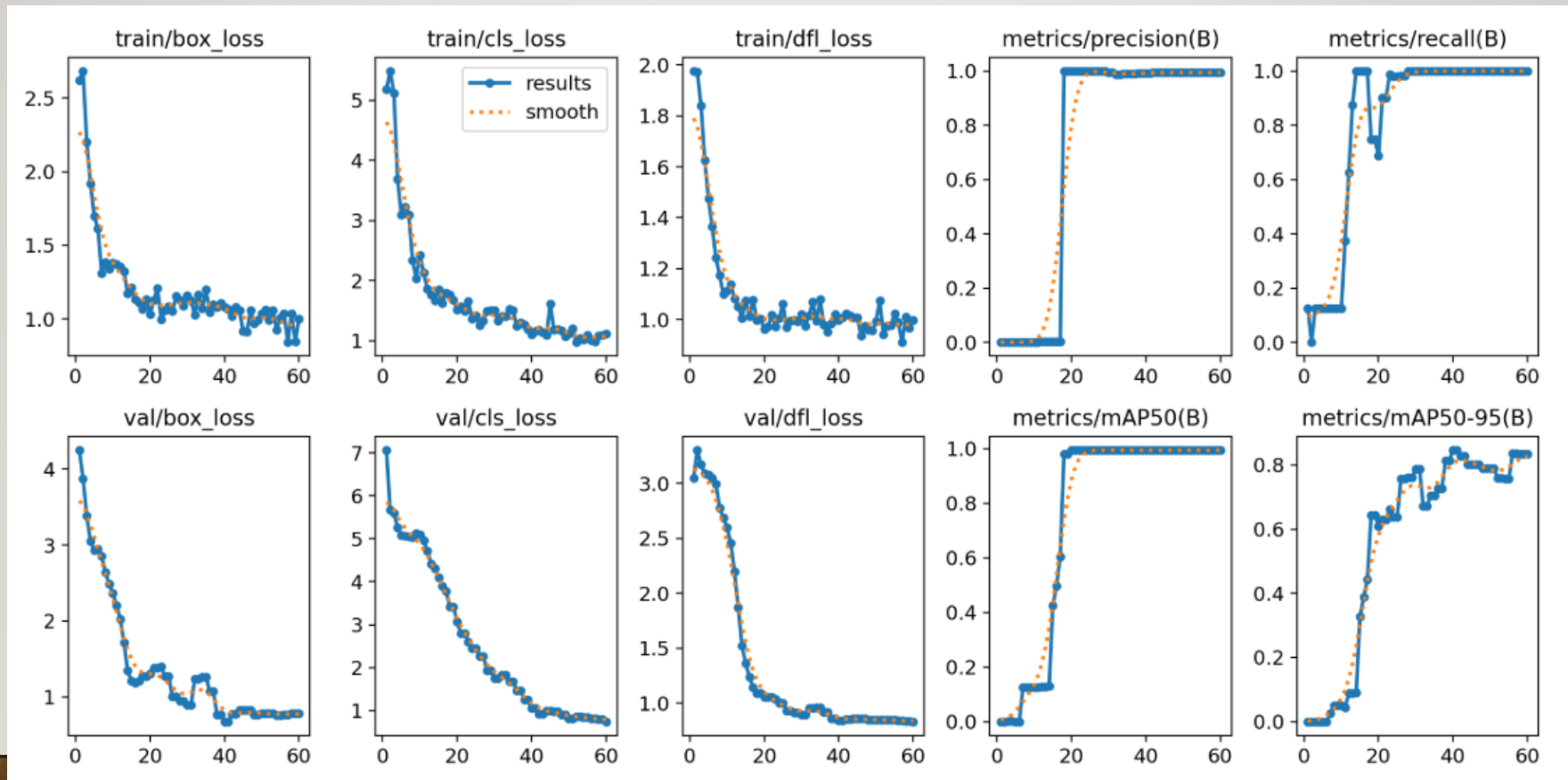
mAP



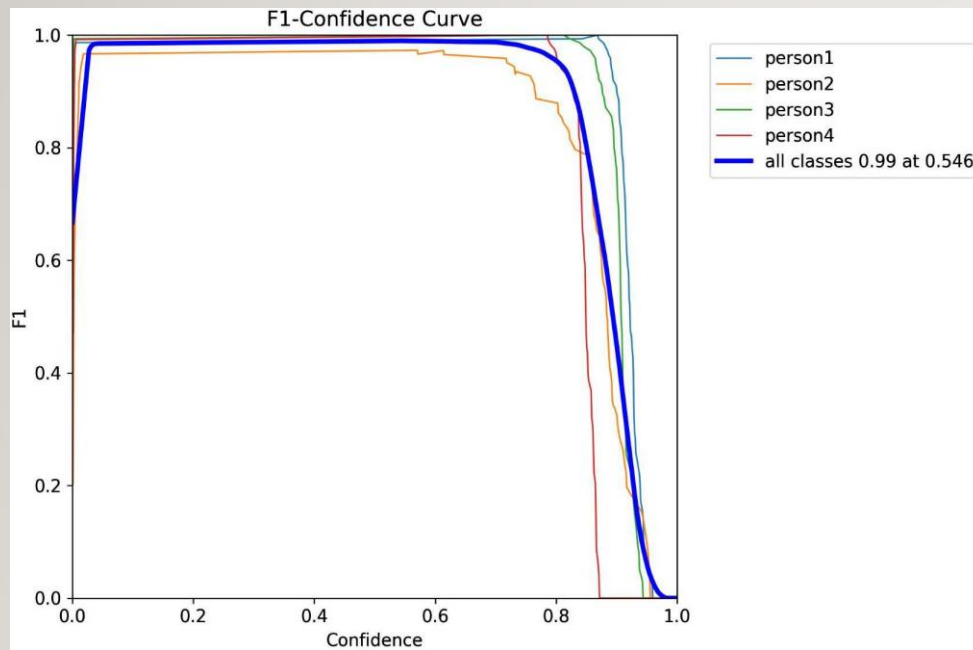
$$AP = \int_0^1 P(R) dR$$

- **PR curve** = shape
- **AP** = area under that shape
- **mAP** = average(AP across classes and IoU thresholds)

EXAMPLE OUTPUT



FI CONFIDENCE



Plain meaning

- **FI** = balance between precision and recall

$$F1 = \frac{2PR}{P+R}$$

KEY METRICS TO COLLECT

- Training
 - Training Hyper Parameters
 - # Epochs (ending)
 - Total Training Time
- Validation
 - mAP50
 - Precision/Recall
 - F1 Confidence
- Test Inference
 - % Correct Predictions
 - Inference speed
 - Average Confidence

CODING HINTS

- Image Capture
- Data Labelling
- Preprocessing
- Yolo8 Object Det (Analyze)



```
2_3_a_create_data_dirs.py
2_3_b_move_image.py
2_3_c_move_labels.py
2_4_a_yolov8_obj_det_ak.ipynb
2_4_b_gpu_test.py
2_4_c_compare_yolo.py
```

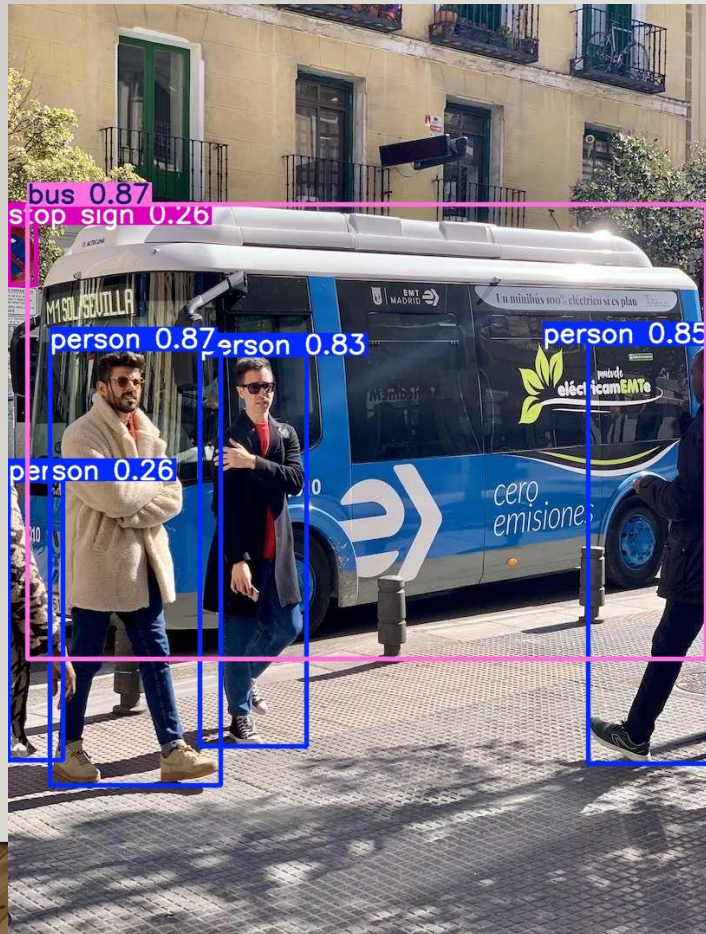
INFERENCE(WEBCAM)

- Image Capture
- Data Labelling
- Preprocessing
- Yolo8 Object Det (WEBCAM)

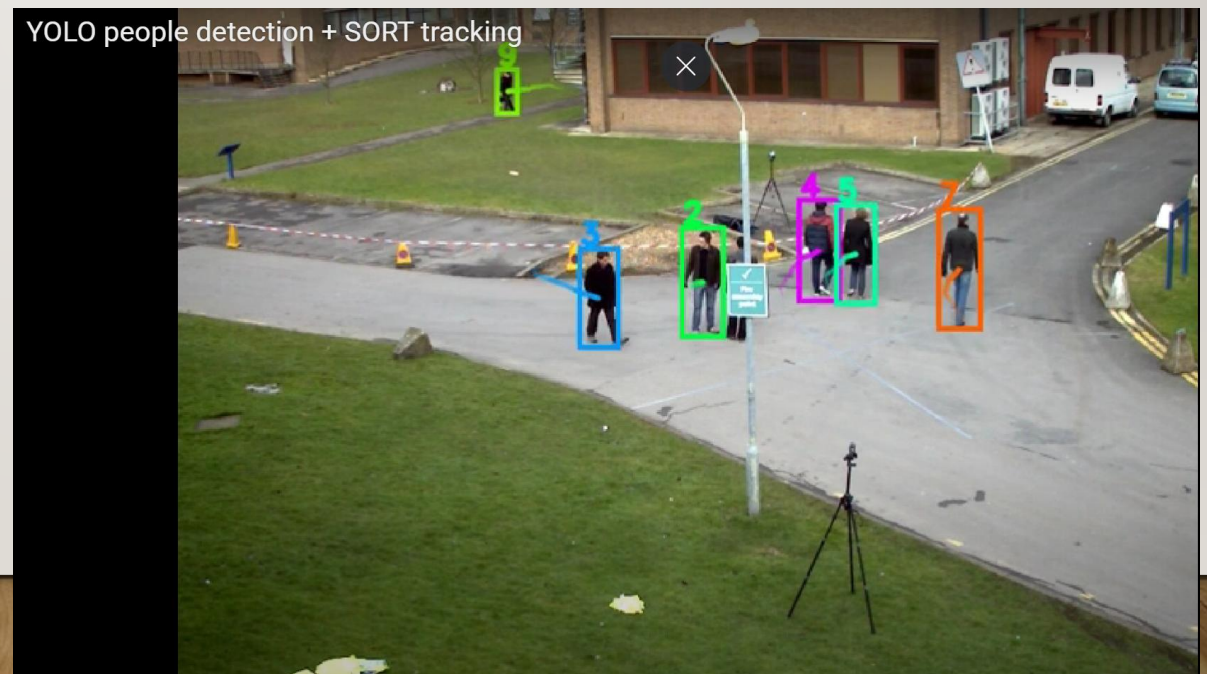


```
2_4_a_yolov8_obj_det_ak.ipynb
2_4_b_gpu_test.py
2_4_c_compare_yolo.py
2_4_d_yolov8_obj_det_wc.py
2_4_e_yolo_publisher_wc.py
2_4_f_yolo_subscriber_wc.py
```

YOLO OBJ. DET. VS. YOLO TRACKING



- [Track - Ultralytics YOLO Docs](#)
 - [\(469\) YOLO people detection + SORT tracking – YouTube](#)
 - [Bing Videos](#)



INFERENCE(AMR)

- Image Capture
- Data Labelling
- Preprocessing
- Yolo8 Object Det (AMR)

 2_4_a_yolov8_obj_det_ak.ipynb 2_4_b_gpu_test.py 2_4_c_compare_yolo.py 2_4_d_yolov8_obj_det_wc.py 2_4_e_yolo_publisher_wc.py 2_4_f_yolo_subscriber_wc.py 2_4_g_yolov8_obj_det.py 2_4_h_yolov8_obj_det_thread.py 2_4_i_yolov8_obj_det_track.py

OBJECT DETECTION EXERCISE

Create a version of real time yolo inference code that publish a ROS topic with annotated image of detection results and view it with rqt or rviz

GETTING DISTANCE VALUE FROM DEPTH IMAGE

DAY 2 - AI VISION (YOLO)

Aa 이름

⚙ status

● Homework_삐뽀삐뽀 소리내기 노드 만들기

● 완료

● RGB Camera

● 완료

● Depth Camera

● 완료

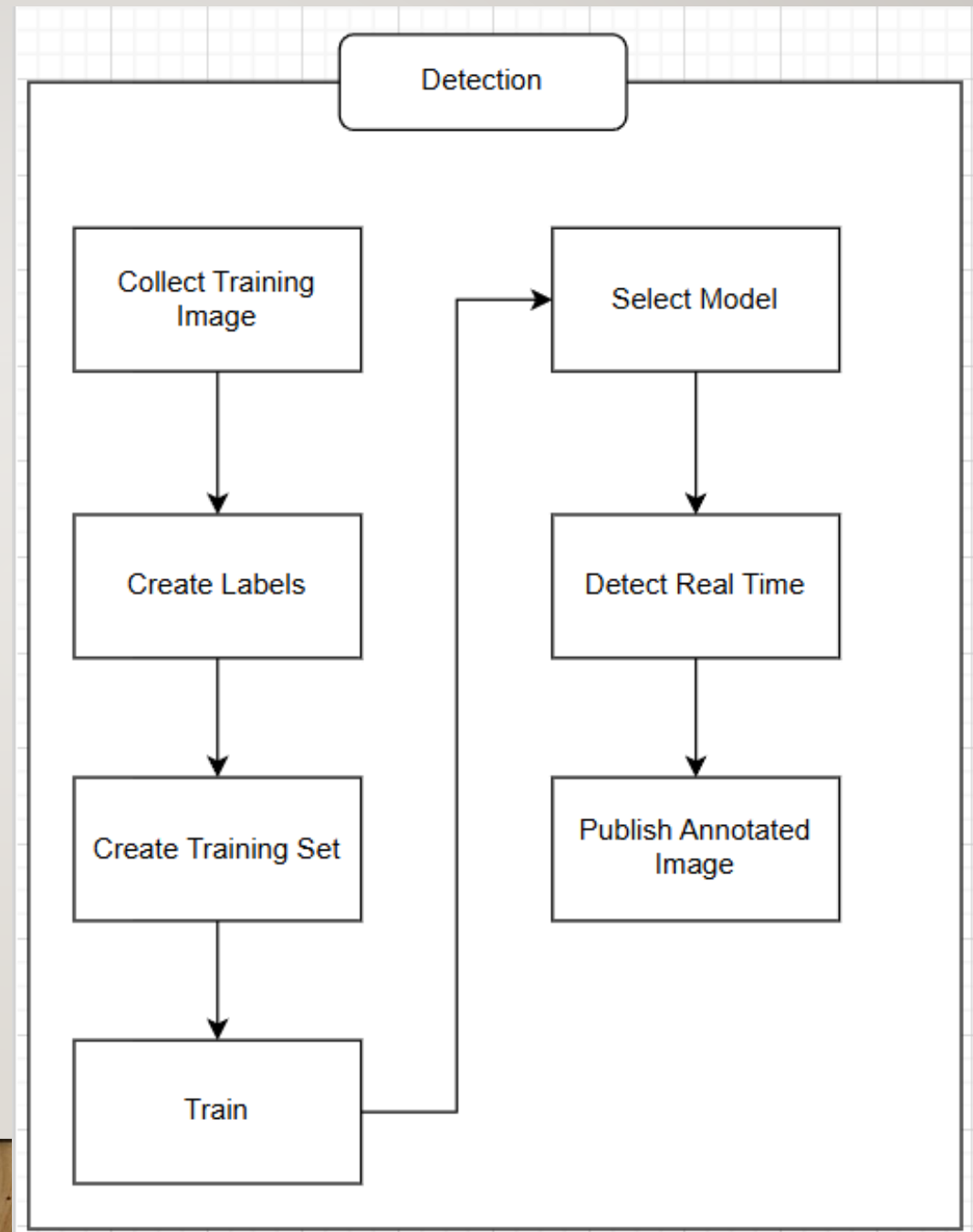
● Robot_Depth

● 완료

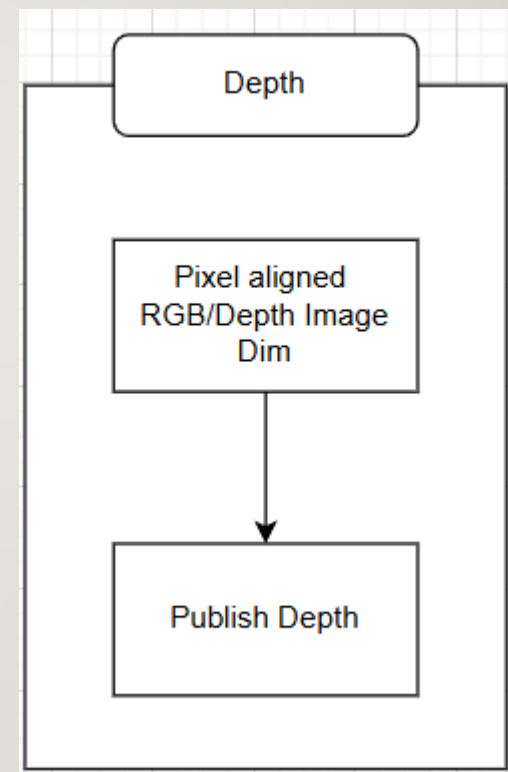
HOMEWORK

- **Achieve aligned RGB & Depth FOV**
- Object Detection
 - Collect various datasets (i.e. different topics/images sizes)
 - Create various models (i.e. v5, v8, v11, etc; arg: Epoch, Batch, Imgsz, augmentation, etc)
 - Analyze the results
 - Determine using key metrics which model best fit your solution
 - Using .pt file to predict/inference on pc
 - **Successfully publish the annotated image topic**
- Depth
 - **Find and display the distance to the center of the detected objects**
- Update System Requirement

TASK TO BE PERFORMED

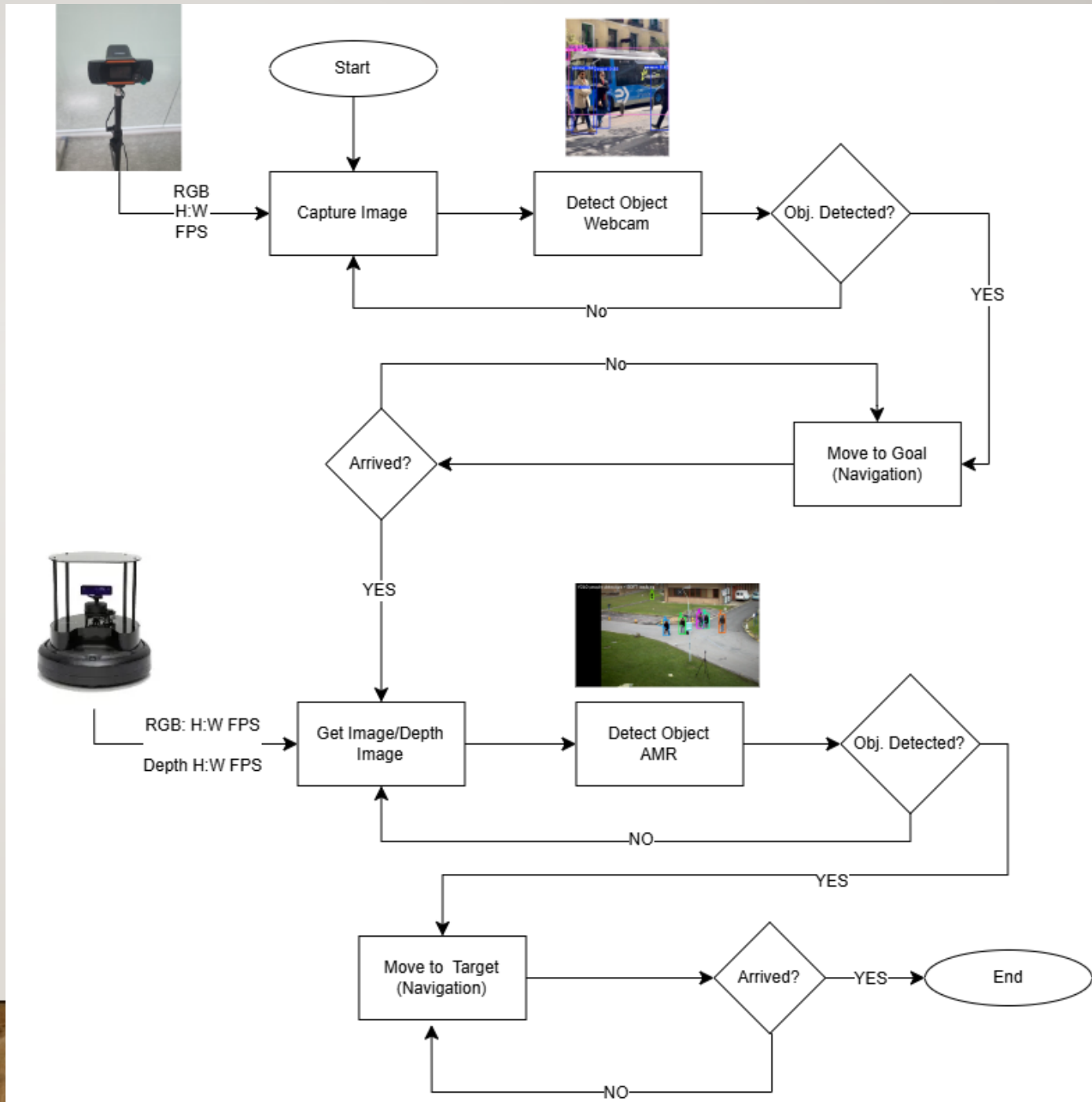


TASK TO BE PERFORMED



UPDATED FUNCTIONAL FLOW DIAGRAM I

Includes selected detection model
and extended logical details



BEFORE GOING HOME (VERY IMPORTANT)!!!

Remember to:

- Dock your robot
- Perform **soft** shutdown by
 - PC Terminal
`$ ssh ubuntu@<AMR IP>`
 - AMR Terminal
`$ sudo shutdown now`

Or

- Perform **soft** shutdown by
 - PC Terminal
`$ ssh ubuntu@<AMR IP>`
 - AMR Terminal
`$ sudo shutdown now`
- Undock your robot
- Power off Create 3



프로젝트 RULE NUMBER ONE!!!

Are we still having
FUN!

